

```
#!/bin/bash

#
=====
===
#      FRP/GRE TUNNEL MANAGER
#      (Personalized Build)
#
=====
===

# --- GRE TAP Configuration ---
GRE_TAP_IF="gt-l2"
BR_IF="br-l2"
IRAN_TUN_IP="10.10.10.2"
FOREIGN_TUN_IP="10.10.10.1"
RT_TABLE_ID="200"
RT_TABLE_NAME="l2tunnel"
MTU_VAL="1400"
MSS_VAL="1360"
GRE_SERVICE="/etc/systemd/system/l2tunnel.service"
GRE_CONFIG="/etc/l2tunnel.conf"

require_root() {
    [[ $EUID -ne 0 ]] && { echo "❌
Error: Run as root"; exit 1; }
}
```

```

# --- Shared Optimization ---
smart_optimize() {
    echo "🔍 Analyzing and Optimizing
System..."
    modprobe nf_conntrack 2>/dev/null
|| true
    modprobe ip_gre 2>/dev/null || true
    modprobe gretap 2>/dev/null || true

    if [ -f
/proc/sys/net/netfilter/nf_conntrack_max ]; then
        local MEM_TOTAL=$(free -m | awk
'/^Mem:/{print $2}')
        if [ "$MEM_TOTAL" -gt 512 ];
then
            sysctl -w
net.netfilter.nf_conntrack_max=131072
>/dev/null
            sysctl -w
net.netfilter.nf_conntrack_buckets=3276
8 >/dev/null
            sysctl -w
net.netfilter.nf_conntrack_tcp_timeout_
established=300 >/dev/null
            sysctl -w
net.netfilter.nf_conntrack_tcp_timeout_
time_wait=30 >/dev/null
            sysctl -w
net.netfilter.nf_conntrack_tcp_timeout_

```

```

close_wait=30 >/dev/null
    fi
fi
    sysctl -w net.ipv4.ip_forward=1
>/dev/null
    sysctl -w
net.ipv4.conf.all.rp_filter=0
>/dev/null
    sysctl -w
net.ipv4.conf.default.rp_filter=0
>/dev/null
}

#
=====
===
# PART 1: GREYAP L2 LOGIC
#
=====
===

cleanup_greyap() {
    echo "🧹 Cleaning up GREYAP..."
    systemctl disable l2tunnel --now
2>/dev/null
    iptables -t nat -F; iptables -t
mangle -F
    ip rule del table $RT_TABLE_NAME
2>/dev/null || true
    ip route flush table $RT_TABLE_NAME
2>/dev/null || true

```

```

        ip link del $GRETAP_IF 2>/dev/null
    || true
        ip link del $BR_IF 2>/dev/null ||
true
        rm $GRE_SERVICE $GRE_CONFIG
2>/dev/null
}

setup_l2_gre() {
    local LOCAL_IP=$1; local
REMOTE_IP=$2; local BRIDGE_IP=$3
    local PUB_IF=$(ip route get 1.1.1.1
| awk '{print $5; exit}')
    ip link add $GRETAP_IF type gretap
local $LOCAL_IP remote $REMOTE_IP dev
$PUB_IF
    ip link add $BR_IF type bridge
    ip link set $GRETAP_IF master
$BR_IF
    ip addr add $BRIDGE_IP/30 dev
$BR_IF
    ip link set $GRETAP_IF mtu $MTU_VAL
up
    ip link set $BR_IF mtu $MTU_VAL up
}

setup_iran_gre() {
    local PORTS=$1
    if ! grep -q "$RT_TABLE_ID
$RT_TABLE_NAME"
/etc/iproute2/rt_tables; then

```

```

        echo "$RT_TABLE_ID
$RT_TABLE_NAME" >>
/etc/iproute2/rt_tables
    fi
    ip route add default via
$FOREIGN_TUN_IP dev $BR_IF table
$RT_TABLE_NAME 2>/dev/null || true
    ip rule add from $IRAN_TUN_IP table
$RT_TABLE_NAME priority 100 2>/dev/null
|| true
    iptables -t mangle -A FORWARD -p
tcp --tcp-flags SYN,RST SYN -j TCPMSS -
-set-mss $MSS_VAL
    for p in $(echo "$PORTS" | tr ',' '
'); do
        p=$(echo $p | xargs); [[ -z
"$p" ]] && continue
        iptables -t nat -A PREROUTING -
p tcp --dport $p -j DNAT --to-
destination $FOREIGN_TUN_IP
        iptables -t nat -A PREROUTING -
p udp --dport $p -j DNAT --to-
destination $FOREIGN_TUN_IP
    done
    iptables -t nat -A POSTROUTING -o
$BR_IF -j MASQUERADE
}

setup_foreign_gre() {
    local PUB_IF=$(ip route get 1.1.1.1
| awk '{print $5; exit}')

```

```

        iptables -t mangle -A FORWARD -p
tcp --tcp-flags SYN,RST SYN -j TCPMSS -
-set-mss $MSS_VAL
        iptables -t nat -A POSTROUTING -o
$PUB_IF -j MASQUERADE
    }

run_gretap_menu() {
    while true; do
        clear
        echo "🟦 GRETAP L2 TUNNEL
SECTION"
        echo "-----"
        _"
        echo "1) Setup IRAN Server"
        echo "2) Setup FOREIGN Server"
        echo "3) Remove GRETAP ONLY"
        echo "4) Back to Main Menu"
        read -p "Selection: " gchoice
        case $gchoice in
            1|2)
                read -p "Local IP: "
LOCAL; read -p "Remote IP: " REMOTE
                [[ $gchoice == 1 ]] &&
read -p "VPN Ports (e.g. 443,9010): "
PORTS
                cleanup_gretap;
smart_optimize
                if [[ $gchoice == 1 ]];
then
                    setup_l2_gre $LOCAL

```

```

$REMOTE $IRAN_TUN_IP
                                setup_iran_gre
"$PORTS"
                                MODE="IRAN"
                                else
                                setup_l2_gre $LOCAL
$REMOTE $FOREIGN_TUN_IP
                                setup_foreign_gre
                                MODE="FOREIGN"
                                fi
                                read -p "Enable
autostart? (y/n): " PERSIST
                                if [[ $PERSIST == "y"
]]; then
                                echo -e
"MODE=$MODE\nLOCAL=$LOCAL\nREMOTE=$REMO
TE\nPORTS='$PORTS'" > $GRE_CONFIG
                                cat <<EOF >

$GRE_SERVICE
[Unit]
Description=L2Tunnel GRE-TAP Service
After=network.target
[Service]
Type=oneshot
ExecStart=$(readlink -f "$0") --start-
gre
RemainAfterExit=yes
[Install]
WantedBy=multi-user.target
EOF

                                systemctl daemon-

```

```

reload; systemctl enable l2tunnel
fi
    echo "✅ GRETAP
Established!"; read -p "Press Enter..."
;;
    3) cleanup_gretap; read -p
"GRETAP Removed. Press Enter..." ;;
    4) break ;;
esac
done
}

#
=====
===
# PART 2: FRP REVERSE LOGIC
#
=====
===

cleanup_frp() {
    echo "🔧 Removing FRP..."
    systemctl stop frps@server-
3090.service frpc@client-3090.service
2>/dev/null || true
    systemctl disable frps@server-
3090.service frpc@client-3090.service
2>/dev/null || true
    rm -f
/etc/systemd/system/frps@.service
/etc/systemd/system/frpc@.service

```



```

    rm -rf /root/frp; rm -f
    /usr/local/bin/frps /usr/local/bin/frpc
    (crontab -l 2>/dev/null | grep -v
    'pkill -10') | crontab -
    echo "✅ FRP Cleaned."
}

install_frp_server() {
    echo "=== Installing FRP Server
    (Iran) ==="
    curl -L -o /usr/local/bin/frps
    http://81.12.32.210/downloads/frps
    chmod +x /usr/local/bin/frps
    mkdir -p /root/frp/server
    cat > /root/frp/server/server-
    3090.toml <<'EOF'
    bindAddr = ":@"
    bindPort = 3090
    transport.heartbeatTimeout = 90
    transport.maxPoolCount = 65535
    transport.tcpMux = false
    auth.method = "token"
    auth.token = "tun100"
    EOF
    cat <<EOF >
    /etc/systemd/system/frps@.service
    [Unit]
    Description=FRP Server Service (%i)
    After=network.target
    [Service]
    ExecStart=/usr/local/bin/frps -c

```

```

/root/frp/server/%i.toml
Restart=on-failure
RestartSec=10s
LimitNOFILE=infinity
[Install]
WantedBy=multi-user.target
EOF

    systemctl daemon-reload; systemctl
enable frps@server-3090.service --now
    (crontab -l 2>/dev/null | grep -v
'pkill -10' ; echo '0 */3 * * * pkill
-10 -x frpc; pkill -10 -x frps') |
crontab -
}

install_frp_client() {
    echo "=== Installing FRP Client
(Kharej) ==="
    curl -L -o /usr/local/bin/frpc
https://raw.githubusercontent.com/lostst
oul6/frp-file/refs/heads/main/frpc
    chmod +x /usr/local/bin/frpc
    mkdir -p /root/frp/client
    read -p "Enter Iran server address:
" server_addr
    read -p "Enter ports (e.g.
8080,9000-9010): " ports
    ports=${ports:-8080}
    cat > /root/frp/client/client-
3090.toml <<EOF
serverAddr = "$server_addr"

```

```
serverPort = 3090
loginFailExit = false
auth.method = "token"
auth.token = "tun100"
transport.protocol = "tcp"
transport.tcpMux = false
transport.poolCount = 20
{{- range $_, $v :=
parseNumberRangePair "$ports" "$ports"
}}
[[proxies]]
name = "tcp-{{ $v.First }}"
type = "tcp"
localIP = "127.0.0.1"
localPort = {{ $v.First }}
remotePort = {{ $v.Second }}
{{- end }}
EOF
    cat <<EOF >
/etc/systemd/system/frpc@.service
[Unit]
Description=FRP Client Service (%i)
After=network.target
[Service]
ExecStart=/usr/local/bin/frpc -c
/root/frp/client/%i.toml
Restart=on-failure
RestartSec=10s
LimitNOFILE=infinity
[Install]
WantedBy=multi-user.target
```

EOF

```
systemctl daemon-reload; systemctl
enable frpc@client-3090.service --now
(crontab -l 2>/dev/null | grep -v
'pkill -10' ; echo '0 */3 * * * pkill
-10 -x frpc; pkill -10 -x frps') |
crontab -
}
```

```
run_frp_menu() {
    while true; do
        clear
        echo "🟡 FRP REVERSE TUNNEL
SECTION"
        echo "-----"
        _"
        echo "1) Install FRP Server
(Iran)"
        echo "2) Install FRP Client
(Foreign)"
        echo "3) Remove FRP ONLY"
        echo "4) Back to Main Menu"
        read -p "Selection: " fchoice
        case $fchoice in
            1) smart_optimize;
install_frp_server; read -p "FRP Server
Started. Press Enter..." ;;
            2) smart_optimize;
install_frp_client; read -p "FRP Client
Connected. Press Enter..." ;;
            3) cleanup_frp; read -p
```

```

"FRP Removed. Press Enter..." ;;
    4) break ;;
    esac
done
}

# --- Service Autostart for GRETAP ---
if [[ "$1" == "--start-gre" ]]; then
    source $GRE_CONFIG; smart_optimize
    if [[ "$MODE" == "IRAN" ]]; then
        setup_l2_gre $LOCAL $REMOTE
        $IRAN_TUN_IP; setup_iran_gre "$PORTS"
    else
        setup_l2_gre $LOCAL $REMOTE
        $FOREIGN_TUN_IP; setup_foreign_gre
    fi
    exit 0
fi

#
=====
===
# MAIN ROUTER
#
=====
===

require_root
while true; do
    clear
    echo

```

```
"===== "  
    echo "          FRP / GRE TUNNEL  
MANAGER"  
    echo  
    "===== "  
        echo "1) GRE TAP L2 (Bridge Mode)"  
        echo "2) FRP (Reverse TCP Mode)"  
        echo "3) Exit"  
        echo  
    "===== "  
        read -p "Select Tunnel Type: "  
main_choice  
        case $main_choice in  
            1) run_gretap_menu ;;  
            2) run_frp_menu ;;  
            3) exit 0 ;;  
            *) echo "Invalid choice." ;  
sleep 1 ;;  
        esac  
done
```